ĐẶC TẢ LUỒNG CHẠY & TRIỂN KHAI

AI-VIDS

Từ CAN frame đến cảnh báo tại edge.

Hai cấu hình trong một spec: **Ubuntu Linux trên VMware** để hiểu và kiểm chứng code hiện tại; **Raspberry Pi 4** cho kiến trúc production mục tiêu.



Mỗi bước được map về task T01–T38 và block A/B/C/D trên kiến trúc.

Trạng thái phát hành: v1.0 — bản spec để review. V2 là thiết kế cần triển khai và nghiệm thu. Việc tạo PDF không triển khai Pi/production.

Thông tin	Giá trị
Ngôn ngữ / ngày	Tiếng Việt · 01/11/2026
Repository	automotive-edge-ai-ids
Đối tượng đọc	Người review hệ thống, embedded/Linux developer, người triển khai Pi
Bằng chứng hiện có	12 tests local; parity 1.660 windows trên PC; 52 windows từ log VM khớp reference
Điểm chặn production	Model demo payload shift 1/40 window vượt ngưỡng; chưa đủ confirmed ALERT; thiếu kiểm chứng Pi và các chức năng vận hành

ĐỊNH HƯỚNG

Cách đọc và so sánh hai cấu hình

Đọc **trang 3 → 12** để hiểu bản đang chạy. Đọc **13 → 21** để thấy Pi phải khởi động, xử lý và phục hồi thế nào. Trang **22 → 24** map đầy đủ task; trang **25** là bảng lệnh tra nhanh.

Tiêu chí	V1 · Ubuntu trên VMware	V2 · Raspberry Pi production
Mục tiêu	Hiểu code, kiểm Linux và SocketCAN	Giám sát CAN dài hạn với giới hạn và health rõ ràng
Nguồn frame	CSV replay hoặc vcan0	can0 qua adapter/driver đã chốt
Kiến trúc CPU	Theo Ubuntu guest, thường x86_64	Linux ARM64/aarch64, kiểm trên Pi thật
Model	demo.forest, dữ liệu tổng hợp	Model/profile theo lab/xe đích, đã đạt holdout
C++ runtime	Một luồng đồng bộ	Capture/compute/logger tách bằng queue giới hạn — cần bổ sung
Dashboard	Replay subprocess	Read-only live event API — cần bổ sung
Vận hành	Terminal, log file thủ công	Service, retention, health, release/rollback
Bằng chứng được dùng	Log VM không thay benchmark Pi	Testbench + Pi measurements + soak + release review

Quy ước trong tài liệu

- **HIỆN CÓ:** đọc được trong source hoặc có bằng chứng local đã lưu.
- **MỤC TIÊU V2:** yêu cầu thiết kế, chưa được coi là tính năng đã có.
- **TBD:** thông số cần chốt; không tự điền bằng kết quả đo trên PC/VM.
- **% công sức:** ước lượng phân bổ scope, không phải % hoàn thành hay accuracy.

03 · Kiến trúc chung và ranh giới
04 · Thành phần và bốn mode hiện có
05 · VM: từ source tới binary và parity
06 · VM: 12 tests thực sự kiểm gì?
07 · VM replay: thứ tự gọi hàm
08 · Đi xuyên một window trong log của bạn
09 · Event JSON, timestamp và ý nghĩa phép đo
10 · VM vcan: luồng Linux socket thật
11 · VM dashboard và điều kiện kết thúc mốc
12 · Training và artifact dùng chung hai cấu hình
13 · Pi production: kiến trúc mục tiêu
14 · Pi: luồng khởi động từ cấp nguồn tới READY
15 · Pi: đường dữ liệu và giới hạn backlog
16 · Pi: alarm và health là hai trục độc lập
17 · Pi: event, evidence, API và retention
18 · Pi: bundle, filesystem và CLI mục tiêu
19 · Pi: service và tiến trình khởi động
20 · Pi: deploy, promote và rollback
21 · Gate nghiệm thu và điểm chặn release
22 · Map task → code → outcome (01–13)
23 · Map task → code → outcome (14–26)
24 · Map task → code → outcome (27–38)
25 · Tra nhanh: lệnh nào đi qua phần nào?
26 · Nguồn tham chiếu và checklist review

CHUNG · A/B/C/D

Kiến trúc chung và ranh giới



Số block A/B/C/D thống nhất với sơ đồ và WBS đã bàn giao.

> Phạm vi v1 là Classic CAN standard 11-bit, payload 0–8 byte. Extended ID/RTR/error không vào feature ở capture; CAN FD chưa hỗ trợ. V2 phải chốt coverage theo bus thực, không tự xem lưu lượng ngoài scope là normal.

- Đường IDS quan sát và phát cảnh báo; không tự gửi lệnh điều khiển/chặn frame.
- Ứng dụng chỉ đọc chưa bảo đảm controller thụ động ở tầng điện: listen-only phải được adapter/driver hỗ trợ và cấu hình.
- A1/A2/A3 là các nguồn/cấp kiểm thử; không cần chạy cả ba cùng lúc.
- CAN ID không xác thực danh tính ECU. Anomaly score không xác định attacker; spoofing/replay hợp ngữ cảnh có thể không phân biệt được nếu thiếu signal/counter/ngữ cảnh phù hợp.

Cơ chế SocketCAN/interface/listen-only: Linux Kernel — SocketCAN. Các lựa chọn module/queue trong spec là thiết kế dự án.

V1 · CẤU TRÚC CODE

Thành phần và bốn mode hiện có

Thành phần	Dùng ở đâu	Input → output
CMakeLists.txt	T07	Source C++17 → native executable
src/main.cpp	B1–B6 + event stdout	Frame/vector/model → feature/score/state
tools/verify_pi_bundle.py	T15/T36	Manifest + golden files → checksum/parity report
tests/test_pipeline.py	Tests	Input tạm + subprocess → assertions
tools/serve.py + ui/index.html	T20	C++ replay stdout → API → canvas/metrics
tools/prepare.py, trên PC	T12/T32/T33/T35	Synthetic data → train/export/validation

Mode ai-vids	Đầu vào bắt buộc	Chạy qua phần nào?
features INPUT OUTPUT	Raw CAN CSV	read_csv → parse_frame → Extractor; xuất feature CSV
score MODEL FEATURES	Forest + vector CSV	Forest constructor → score; xuất scalar mỗi dòng
replay INPUT MODEL [--realtime]	Raw CSV + forest	Extractor + model + rule + state; xuất NDJSON
capture IFACE MODEL	Linux CAN interface + forest	SocketCAN → cùng core; xuất NDJSON

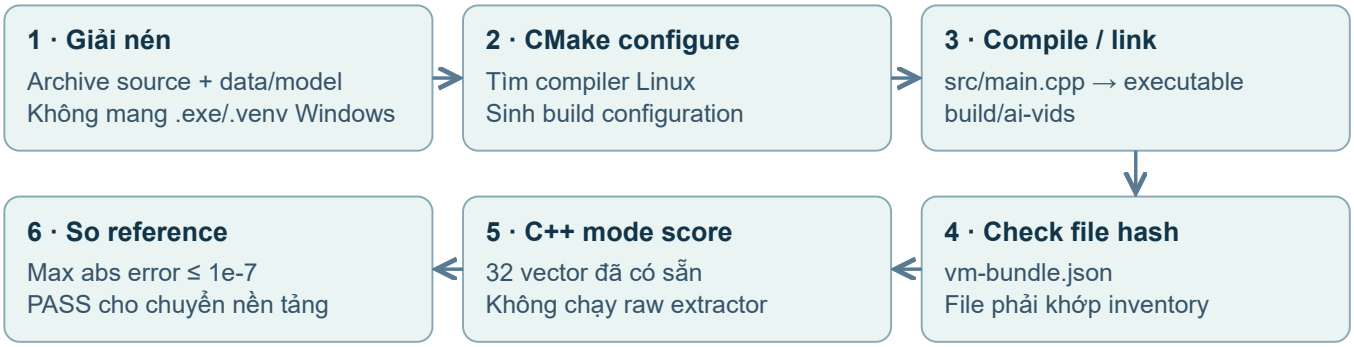
Những thứ cần cài, đúng theo vai trò

- Build: CMake, C++ compiler/build tools. Ubuntu dùng build-essential/cmake.
- Inference terminal: binary + CSV/model hoặc CAN interface. Không cần sklearn.
- Checker/tests/UI: Python 3 standard library. Browser chỉ vẽ kết quả.
- vcan: Linux module vcan, iproute2; cansend/candump thuộc can-utils.
- Training trên PC mới cần numpy/sklearn; không chạy lại khi replay.

Source / trace: CMakeLists.txt:3; src/main.cpp:260; tools/serve.py:19; tools/prepare.py:65

V1 · T36 / T07 / T14 / T15

VM: từ source tới binary và parity




```
cd ~/ai-vids-vm
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j2
python3 tools/verify_pi_bundle.py --manifest vm-bundle.json
```

Hàm thực tế: verifier đọc manifest → hashlib.sha256 từng file → subprocess gọi `ai-vids score` → đọc 32 score → so với golden-scores → in JSON PASS.

> PASS chỉ xác nhận integrity và score parity. Checker không đo recall/false alarm; Ubuntu x86_64 PASS chưa chứng minh ARM64/Pi PASS. Tên file có “pi” được giữ tương thích, không ép kiến trúc.

Source / trace: tools/verify_pi_bundle.py:14; src/main.cpp:271; Forest::score:169; T07/T15 mới hoàn thành phần Linux khi có log đích.

V1 · KIỂM CHỨC NĂNG

VM: 12 tests thực sự kiểm gì?

```
python3 -m unittest discover -s tests -v
```

unittest tạo CSV/model lỗi trong thư mục tạm, gọi executable C++ bằng subprocess và assert output/exit code. Phần lớn tests gọi code thật.

Test / nhiệm vụ	Input / đường code	Điều kiện đạt
Golden features + boundary	00/FF cùng ID; frame ở 100 ms → features	Giá trị tính tay đúng; frame biên sang window mới
Silent gap	Frame 0 rồi 300 ms → replay	Các window giữa là NO_DATA
Label independence	Đổi normal thành attack trong cột label	Kết quả không đổi, bỏ timing đo
DLC mismatch	DLC 8 nhưng payload 1 byte	Reject/exit nonzero
Out-of-order timestamp	Timestamp giảm	Reject
Negative timestamp	Timestamp âm	Reject
Extended ID	ID ngoài standard 11-bit	Reject theo scope v1
Empty payload	DLC 0	Được đọc hợp lệ
Wrong model schema	Header model sai	Forest loader reject
Synthetic scenarios	Replay 46 s	460 windows; xuất hiện reason theo kịch bản
Export report metadata	Đọc validation.json có sẵn	Kiểm metadata; không chấm lại 1.660 vector trên VM
Realtime boundary	Một window 100 ms	Không phát kết quả ngay tại t=0

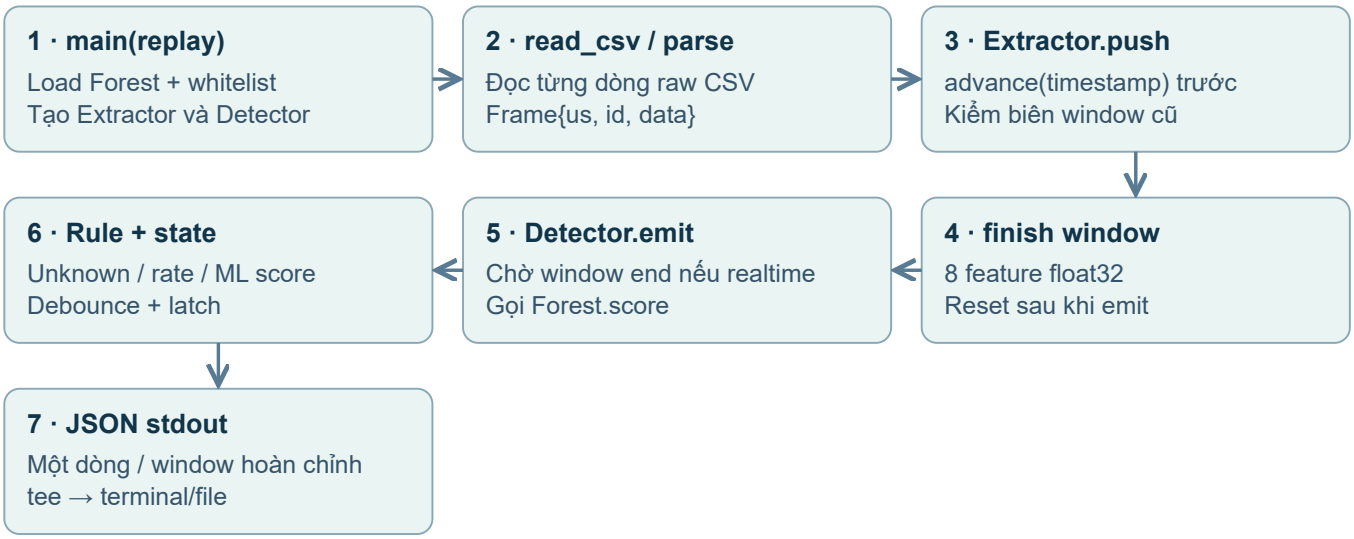
> Tests OK không thay quality evaluation. Scenario test chỉ yêu cầu thấy dấu hiệu; không chứng minh model bắt đủ 40/40 payload windows. State transition coverage cũng chưa đầy đủ.

Source / trace: tests/test_pipeline.py:18; 32; 91; 105; 110

V1 · A1 → B1 → B2 → B3 → B4/B5 → B6

VM replay: thứ tự gọi hàm

```
./build/ai-vids replay data/demo.csv models/demo.forest --realtime
```



Callback/lambda nối các bước trong cùng một luồng, chưa phải queue hoặc RPC.

Arch / task	Hàm	Thứ tự cần nhớ
B1 · T08	parse_frame / read_csv	Label CSV không tham gia quyết định
B2 · T11	advance	Đóng window cũ trước khi thêm frame ở biên
B3 · T12	push + finish	Tích lũy từng frame; tạo vector lúc đóng window
B4/B5/B6	score + emit	Một lần model/window, không phải model/frame

Source / trace: src/main.cpp:279 → 122 → 98 → 76 → 66 → 46 → 190 → 169 → 206

V1 · VÍ DỤ 40,8 → 40,9 GIÂY

Đi xuyên một window trong log của bạn

92 frame → 8 feature → 64 cây → score 0,461507 → không kích hoạt rule → HEALTHY. Một dòng log là một window 100 ms.

Index / feature	Giá trị tái dựng	Ý nghĩa
0 · fps	920	92 / 0,1 giây = 920 frames/s
1 · unique_ids	3	Số ID khác nhau
2 · id_entropy	1.584791422	Độ phân bố ID theo -Σ p log2 p
3 · max_id_share	0.3369565308	Tỷ trọng ID xuất hiện nhiều nhất
4 · byte_mean	121.0584259	Trung bình giá trị byte payload
5 · byte_std	19.05335999	Population standard deviation
6 · bit_flip_mean	0.3362008333	Tỷ lệ bit đổi giữa frame cùng ID/cùng DLC
7 · same_id_iat_ms	3.260887623	Trung bình inter-arrival cùng ID trong window

Vector được tái dựng từ CSV/model gốc, không phải feature được in trong paste. 52/52 score ở đoạn log đầy đủ khớp reference trong 1e-9; khác biệt tối đa khoảng 4,67e-11 do số chữ số in.

```
unknown_frames = 0   → hard rule ID không kích hoạt
fps = 920 <= 1800    → hard rule rate không kích hoạt
score = 0.461507 < threshold = 0.608136
abnormal = false    → reason = normal → state = HEALTHY
```

Trong cây đầu: IAT 3,260888 > 2,908064 nên đi phải; byte_std 19,05336 ≤ 19,17534 nên đi trái. Mỗi cây đi tới leaf; tổng hợp path length cho ra anomaly score, không phải xác suất tấn công.

Source / trace: runtime/vm-log-walkthrough.json; Extractor::finish:46; Forest::score:169; Detector::emit:190

CHUNG · CONTRACT V1

Event JSON, timestamp và ý nghĩa phép đo

Trường	Nguồn trong code	Cách đọc
t_ms	w.start / 1000	Bắt đầu window trong miền thời gian của source
window_end_ms	(start + WINDOW_US) / 1000	Kết thúc window; không phải UTC
frames / fps	w.count / w.x[0]	Số frame và rate quy đổi trên 100 ms
score / threshold	Forest.score / model header	Score lớn hơn threshold là ML anomaly
state / reason	Decision engine	State có lịch sử; reason của window hiện tại
unknown_frames	Extractor + whitelist	Frame ngoài whitelist trong window
inference_us	steady_clock quanh score()	Chỉ model; loại trừ window/feature/rule/UI
rx_drops	SO_RXQ_OVFL ở capture	Socket-drop counter; replay giữ mặc định 0
ignored_frames	Frame filter ở capture	Frame ngoài scope; replay giữ mặc định 0

Bốn đại lượng cần đo riêng

- **Model latency:** riêng thời gian score(x); ví dụ log là 20,126 µs = 0,020126 ms.
- **Processing latency:** từ frame/window đủ điều kiện tới quyết định, gồm queue/feature/model/rule.
- **Detection delay:** từ attack onset ground truth tới first confirmed alert; gồm pha window và debounce.
- **UI delay:** thời gian alert đã có tới lúc người dùng thấy; UI v1 poll mỗi 350 ms.

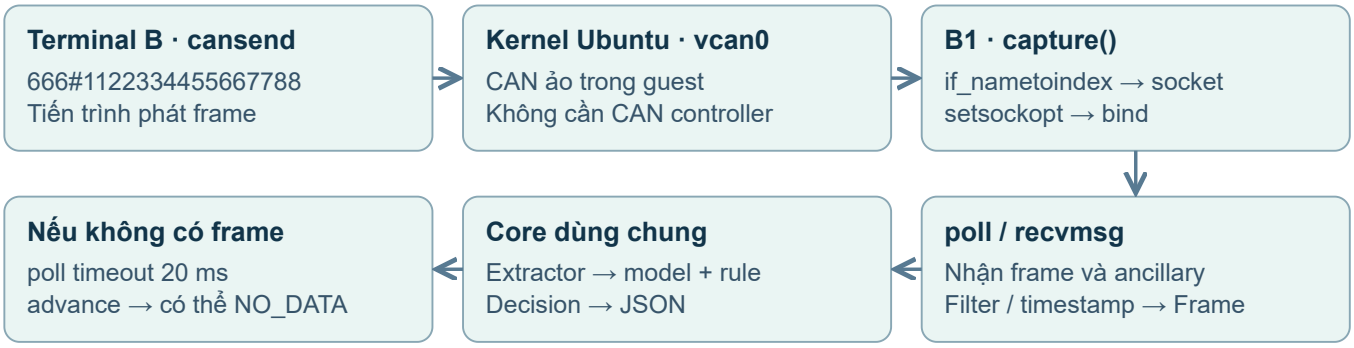
> Không suy ra detection delay từ model P95. Counter 0 trong replay không chứng minh CAN vật lý không mất frame. VM timing còn chịu ảnh hưởng host/hypervisor.

Source / trace: src/main.cpp:182–212; ui/index.html:24–25. V2 cần timestamp source/receive/decision có clock_domain.

V1 · T03 / T09 · A2 → B1

VM vcan: luồng Linux socket thật

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set dev vcan0 up
./build/ai-vids capture vcan0 models/demo.forest
```




```
cansend vcan0 666#1122334455667788
```

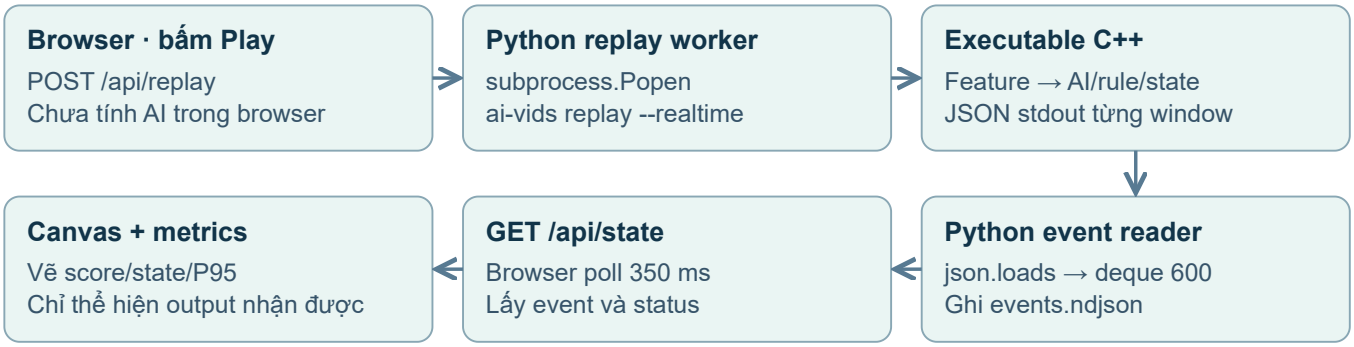
0x666 ngoài whitelist nên hard rule tạo unknown_id/ALERT tại cuối window dù score AI thấp. Capture không đọc demo.csv; timestamp hiện là monotonic user-space receipt time từ lúc capture khởi động.

> vcan kiểm API/logic nhận Linux; không mô phỏng arbitration, lỗi điện hoặc tải CAN vật lý. Một frame ID whitelist gửi tay cũng không tạo đúng phân bố normal training.

Source / trace: src/main.cpp:218–255. Linux Kernel: SocketCAN/vcan.

V1 · T20 / C1

VM dashboard và điều kiện kết thúc mốc




```
python3 tools/serve.py --port 8765
```

Mở browser trong guest tại `http://127.0.0.1:8765`. Muốn xem từ host dùng SSH tunnel. Capture terminal không tự đi vào server v1; live UI thuộc T21.

Bằng chứng kết thúc VM	Điều kiện
Environment + build log	Ghi OS/kernel/arch/compiler; native build thành công
verify-vm.json + tests-vm.log	32 golden score PASS; 12 tests OK nếu đã chạy
replay-vm.ndjson	460 windows cho full demo; paste hiện chỉ có 52 window cuối đầy đủ
capture-vcan.ndjson + stderr	Có frame từ cansend, unknown_id và NO_DATA phù hợp

Validation metadata/nhãn “local PC” trong UI v1 có phần cố định. Đọc host/log của lượt chạy, không xem label đó là tự nhận diện VM/Pi. Python http.server là công cụ review; không dùng nguyên server này làm production API.

Source / trace: tools/serve.py:19/63/79/90; ui/index.html:22–25

CHUNG · D1 → D2 → D3 → D4 → B4

Training và artifact dùng chung hai cấu hình



Hiện có trong demo	Yêu cầu cho production
64 cây; max_samples 128; 8 feature	Candidate phải đạt quality trên dữ liệu đại diện
threshold = quantile(-score_samples(val), 0.995)	Ngưỡng chọn trên validation theo false-alarm budget đã chốt
Whitelist 0x100/0x200/0x300; rate >1800	Profile đúng bus/xe và version; không mang nguyên ngưỡng toy
Window 100 ms; không scaler	Manifest ràng buộc schema/window/preprocessing tương thích
Header + SHA-256 + metadata demo	Runtime/profile/data/model versions; xác thực nguồn release theo kênh phân phối

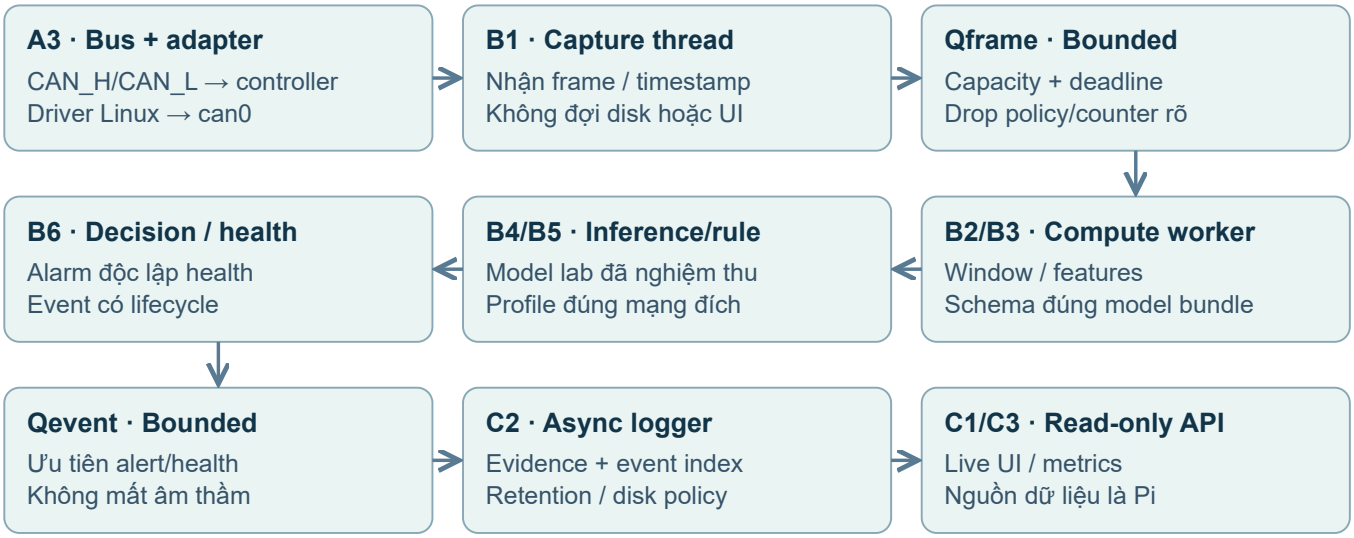
> Model demo chưa đạt payload-shift detection bền vững: 1/40 windows vượt ngưỡng, chưa confirmed ALERT. Không promote artifact này thành model production.

Quy ước score dự án là `-sklearn.score_samples(x)`; không nhầm decision_function/offset. Nguồn: IsolationForest API.

Source / trace: tools/prepare.py:75/81/84/86/96/119; T28–T35.

V2 · A3/A4 + B/C · CẦN TRIỂN KHAI

Pi production: kiến trúc mục tiêu



Màu/khối mô tả V2 target; queues/logger/live API chưa có trong runtime hiện tại.

- Pi: Linux 64-bit; binary ARM64 build/kiểm trên đúng OS/libc; không copy executable Windows.
- Adapter có controller + transceiver + driver hỗ trợ chế độ được chốt. Termination ở hai đầu bus; phương án reference/cách ly theo adapter.
- Bitrate, ID format và coverage phải xuất phát từ bus thực. Không mặc định mọi bus đều 500 kbit/s.
- Nguồn/tản nhiệt/storage theo Pi thực; phải đo throttle và write load trước vận hành dài hạn.

> Production là đích của thiết kế này. Hiện chưa có bằng chứng chạy Pi, queue/logger/service/rollback hoặc chất lượng model đạt production.

V2 · BOOT / READINESS

Pi: luồng khởi động từ cấp nguồn tới READY



Readiness requirement	Điều kiện
PI-BOOT-01	Đúng model/runtime/schema/profile; checksum/self-test đạt; đường lưu trữ khả dụng
PI-BOOT-02	Interface tồn tại và cấu hình đúng; mode/coverage khớp mạng đã chốt
PI-BOOT-03	Capture/compute/logger heartbeat tiến triển; không backlog quá deadline
PI-BOOT-04	Quan sát/không dữ liệu được phân biệt theo profile, không báo healthy vì model chưa load

Nếu model corrupt hoặc schema sai: không READY; phát lỗi rõ và dừng/chờ can thiệp theo policy. Nếu bus im lặng hợp lệ, dùng trạng thái NO_DATA/IDLE được khai báo, không tự xác nhận mạng an toàn.

Source / trace: T07/T09/T15/T19/T35/T38. Hiện main.cpp chỉ load + bind/capture; quy trình readiness V2 cần bổ sung.

V2 · THỜI GIAN CHẠY

Pi: đường dữ liệu và giới hạn backlog

Bước	Input → output	Ràng buộc thiết kế
1. Capture	can_frame + timestamp → normalized Frame	Không gọi disk/UI; quan sát kernel/driver/socket drops
2. Qframe	Frame → compute worker	Bounded; counter; deadline; không tự tăng RAM
3. Window/feature	Frames → vector + data_quality	Order/clock đúng; cửa sổ có mất dữ liệu phải degraded
4. Model/rule	Vector/profile → score + reasons	Không nạp/train model mới trong hot path
5. Decision	Evidence → alarm transition + health	Lưu model/profile versions và thời điểm decision
6. Qevent/logger	Event → durable index/raw chunks	Giới hạn tải; alert/health không bị bỏ âm thầm
7. API/UI	Store/snapshot → read-only view	UI không tạo backpressure vào capture

Định nghĩa ngân sách trước khi chọn capacity

```
capacity cơ sở ≈ measured_peak_frames_per_s × max_queue_wait_s
cộng headroom được review; vẫn phải có max age/deadline
detection_delay = decision_time - attack_onset_time
```

- Chốt peak load theo bus/capture thực; số frame/s không tự suy ra chính xác từ bitrate.
- Overflow policy phải rõ (ví dụ drop-new); tính đúng số rơi và đánh dấu window bị ảnh hưởng.
- Không chỉ đo average: cần P95/P99, max backlog age và resource dưới tải/ghi log chậm.
- Cả feature schema, window/hop và debounce phải được tính vào độ trễ phát hiện.

> SPSC queue, deadline/drop policy và worker tách luồng là yêu cầu mới của T26, chưa tồn tại trong C++ hiện tại.

Source / trace: T10/T11/T12/T19/T25/T26; thay đổi concurrency phải kiểm parity/order/lifetime bộ nhớ.

V2 · T18 / T19 / T27

Pi: alarm và health là hai trục độc lập

Alarm state

```
CLEAR → SUSPECT → OPEN
OPEN → UPDATED → CLOSED
Mỗi alarm có ID/lifecycle
và evidence/model versions
```

Debounce/hysteresis theo profile; rule nghiêm trọng có thể mở ngay sau window. Không đếm mỗi window OPEN thành một attack event.

Sensor health

```
STARTING → OK
OK → STALE / DEGRADED
bất kỳ → FAILED
phục hồi → OK + audit event
```

Silence, drops, interface lỗi, disk/backlog và heartbeat quyết định health; không tự đóng alarm đang mở.

Sự cố	Hành vi yêu cầu	Nghiệm thu
Mất traffic	STALE/NO_DATA theo profile; giữ alarm	Silence không xóa lịch sử alert
Interface down	Health FAILED/STALE; lỗi có ngữ cảnh	Không in normal giả; recovery quan sát được
Socket/app queue drop	DEGRADED; đếm đúng; mark window	Mất dữ liệu không bị che bởi score normal
Model/schema lỗi	Không READY; không âm thầm bỏ ML	Fail rõ; rules-only chỉ khi cấu hình và hiển thị
Disk full/slow	Health/storage error; policy evidence	Capture không treo theo disk
Worker heartbeat đứng	Health FAILED; supervisor dừng với exit nonzero theo deadline đã chốt	Gây treo worker; restart không che downtime
Process restart	Boot/session mới; phục hồi/open alarm policy	Không làm mất dấu downtime hoặc alarm

> V1 hiện có một state chung; NO_DATA reset good/bad và thay state. Vì vậy phải sửa T19 trước khi coi là health/alarm production độc lập.

Source / trace: Current: Detector::emit, main.cpp:198. Target: T19/T22/T27.

V2 · C1/C2/C3

Pi: event, evidence, API và retention

Đối tượng	Trường/đầu ra tối thiểu	Dùng để
Window event	window start/end, score, reasons, schema/model/profile, data_quality	Tái hiện một quyết định
Alert event	alert_id, opened/updated/closed, IDs liên quan, severity/reasons	Đếm và theo dõi một incident
Health event	interface/drop/backlog/storage/heartbeat, boot/session	Phân biệt detector lỗi với mạng bất thường
Evidence chunk	Raw pre/post-window, clock metadata, event reference, checksum	Đi từ alert về frame nguồn
Read-only API	Live event cursor, health/readiness, metrics	UI/operator quan sát nguồn Pi
Audit event	Release activation/rollback, profile/threshold/config changes	Truy nguyên thay đổi vận hành

Storage contract

- Chốt quota byte/time và reserve cho health/error; retention không phải “giữ mãi”.
- Raw pre/post-event buffer và durable event index phải bounded; ghi độc lập capture.
- Định nghĩa flush/fsync theo mức mất dữ liệu chấp nhận; test sudden restart/disk-full.
- stdout/journal dùng cho chẩn đoán process; không thay thế evidence store có lifecycle.

API contract đề xuất, chưa có trong server v1

```
GET /healthz       → tiến trình/worker còn sống?
GET /readyz        → đủ điều kiện phân tích có ý nghĩa?
GET /api/events?after=<cursor> → sự kiện live từ store
GET /api/metrics   → thời gian, drop, resource, release
```

Mặc định local-only hoặc truy cập qua SSH tunnel; nếu expose mạng phải có authentication/TLS và quyền đọc phù hợp. UI production không có nút phát scenario lên CAN.

Source / trace: T21/T22/T23/T25. Server http.server hiện tại chỉ là review tool; API trên là target mới.

V2 · D4 / T35 / T36

Pi: bundle, filesystem và CLI mục tiêu

```
/opt/ai-vids/releases/<release_id>/
  bin/ai-vids
  model/model.forest
  model/manifest.json
  profile/profile.json
  golden/features.csv + scores.txt
/opt/ai-vids/current -> releases/<release_id>
/etc/ai-vids/config.json
/var/lib/ai-vids/         # events / evidence / state
/run/ai-vids/             # runtime heartbeat / sockets
```

Manifest/config contract	Yêu cầu
Release identity	runtime build/arch/OS-ABI + model_id + schema_id + profile_id
Compatibility	Feature order/units/window/scaler/threshold khớp binary; golden parity
Data provenance	Training-data fingerprint, split/evaluation report và production eligibility
Interface/profile	can0, bitrate/mode do setup kiểm; coverage ID/FD đúng scope
Runtime budget	Queue capacity/deadline, silence policy, storage quota từ bench đã chốt
Integrity/authenticity	Hash kiểm integrity; xác thực publisher theo kênh release, không gọi hash là chữ ký

CLI V2 đề xuất: `ai-vids run --config /etc/ai-vids/config.json`. Mode `run`, config parser, manifest enforcement và worker supervisor **chưa được implement**. Binary hiện tại chỉ có features/score/replay/capture.

Tách lệnh mới để không giả vờ cấu hình đã được áp dụng. V2 parser phải reject option/schema không hỗ trợ; loader phải chặn demo-only model ở chế độ production.

Source / trace: Current CLI: src/main.cpp:260. Target: T17/T19/T26/T35/T38.

V2 · TEMPLATE CHƯA THỂ DÙNG VỚI BINARY V1

Pi: service và tiến trình khởi động

> Unit dưới đây dành cho daemon V2 sau khi đã có mode run/config. ai-vids-can.service là unit setup adapter/interface phải viết và kiểm với OS/driver thật. Đây không phải installer đã chạy.

```
[Unit]
Description=AI-VIDS production candidate
Requires=ai-vids-can.service
After=local-fs.target ai-vids-can.service
StartLimitIntervalSec=60
StartLimitBurst=5

[Service]
Type=simple
User=ai-vids
Group=ai-vids
WorkingDirectory=/opt/ai-vids/current
ExecStart=/opt/ai-vids/current/bin/ai-vids run --config /etc/ai-vids/config.json
Restart=on-failure
RestartSec=3
TimeoutStopSec=10
RuntimeDirectory=ai-vids
StateDirectory=ai-vids
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
ReadWritePaths=/var/lib/ai-vids /run/ai-vids
StandardOutput=journal
StandardError=journal

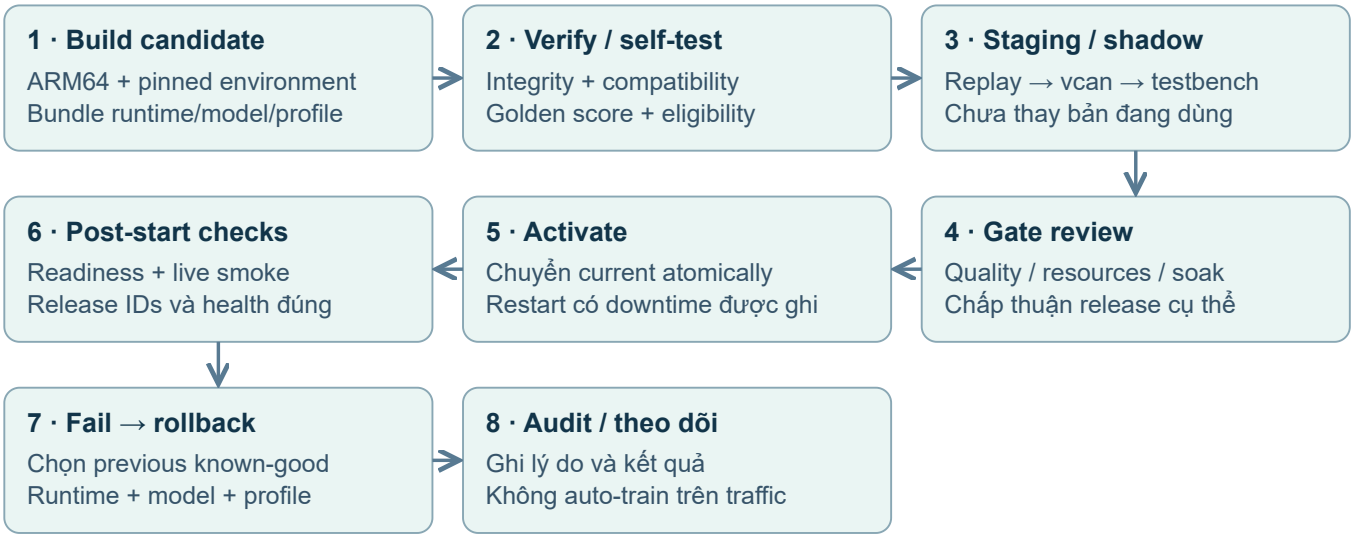
[Install]
WantedBy=multi-user.target
```

- Interface được cấu hình bởi đơn vị quản trị riêng; IDS không tự đổi bitrate trong hot path.
- Type=simple chỉ giám sát process; READY phải do health/readiness contract kiểm. Không thêm Type=notify/WatchdogSec khi chưa có protocol heartbeat tương ứng.
- Không bật PrivateNetwork làm mất network namespace chứa can0. Kiểm quyền/sandbox trên OS đích; không mặc định chạy root để bỏ qua lỗi.

Cơ chế service/restart/type: systemd.service; môi trường/path isolation: systemd.exec. Phải đối chiếu phiên bản systemd thực tế.

V2 · D5 / T37 / T38

Pi: deploy, promote và rollback



Tình huống	Hành vi yêu cầu
Candidate hash/schema sai	Không activate; giữ current; trả lỗi có release ID
Service boot fail / không ready	Không đánh dấu deploy thành công; rollback theo policy
New model tăng false alerts	Giữ evidence và operator review; rollback bản đã biết tốt
Rollback	Đổi cả bộ tương thích; không chỉ thay forest khi feature schema khác
Đang có alarm lúc restart	Giữ/rehydrate lifecycle hoặc mở incident downtime theo policy đã test

Pi staging: kiểm binary bằng CLI hiện có trước V2

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j2
python3 tools/verify_pi_bundle.py
./build/ai-vids replay data/demo.csv models/demo.forest --realtime
```

Sau khi có profile/model lab hợp lệ, capture dùng `ai-vids capture can0 MODEL`. Replay demo trên Pi chỉ kiểm chuyển nền tảng; không activate demo model cho production.

> Production chỉ được thực hiện sau khi bạn chấp thuận release và các gate đã đạt. PDF này không thực hiện SSH, đổi current, cài service hay cập nhật model trên thiết bị.

Source / trace: T24/T25/T27/T34/T35/T37/T38. Source hiện tại chưa có updater, service installer hoặc rollback manager.

CHUNG / V2 · ACCEPTANCE

Gate nghiệm thu và điểm chặn release

Gate	Điều kiện đạt / đề xuất	Bằng chứng
G1 · Linux core	Native build; checksum; golden error ≤1e-7; 12 tests trong scope	VM logs; chưa thay Pi acceptance
G2 · Pi parity	ARM64 binary trên Pi đúng cấu hình; golden score PASS	Pi environment + checker report
G3 · Live input	vcan rồi CAN testbench; clock/order/filter/drop observability	Ground truth và capture logs
G4 · Model quality	Chốt recall/event recall, false alerts/giờ và delay budget trước tuning; holdout độc lập	AI-only/rule-only/fused report; failure cases
G5 · Performance	Mốc thử ban đầu ≥1000 frames/s, model P95 <20 ms; rated bus load và headroom phải chốt theo đo thực	Pi throughput/drop/CPU/RSS/nhiệt/backlog/P95/P99
G6 · Reliability	Queue/disk/interface faults; restart; soak dài hạn. Đề xuất 24 h sau khi thống nhất tải và storage policy	Fault matrix; soak report; không unaccounted loss
G7 · Release	Ready/stop/restart/rollback đúng; artifact compatibility; production approval	Release checklist + audit + quyết định review

Những ngưỡng phải chốt trước final test

- False-alarm budget và recall/detection-delay mục tiêu theo kịch bản; hiện chưa có giá trị được phê duyệt.
- Rated/peak traffic, allowable backlog/drop, CPU/RSS budget, nhiệt/throttle, quota và thời lượng lưu evidence.
- Tiêu chí startup readiness, silence hợp lệ, outage/recovery và rollback triggers.

Release blockers hiện tại: model demo payload shift chưa đạt; thiếu dataset/profile thực; chưa đo Pi; chưa live UI, evidence retention, bounded queues, health/alarm separation hoặc rollback. Build chạy được không vượt qua các gate này.

Source / trace: Mốc ≥1000 frames/s và <20 ms là mục tiêu thử nghiệm ban đầu trong ý tưởng, chưa phải kết quả đo/khả năng đã chứng minh.

TRA CỨU · WBS / ARCH

Map task → code → outcome (01–13)

Task đánh theo vị trí kiến trúc, không phải thứ tự bắt buộc. Một hàm có thể hiện thực nhiều block. % là công sức tương đối; tổng 38 task bằng 100%.

Task / arch / effort	Công việc và implementation hiện tại	Outcome dùng để
T01 · A1 2% · Kiểm local	Chốt hợp đồng dữ liệu CAN parse_frame [98] · schema CSV	Làm đầu vào chung cho replay, capture adapter và xử lý dataset.
T02 · A1 2% · Kiểm local	Tạo dữ liệu và replay có thể lặp lại generate trên PC; main replay [279]	Tái hiện lỗi và review logic bằng cùng một đầu vào.
T03 · A2 2% · Chuẩn bị	Chuẩn bị vcan trên Linux/Pi modprobe / ip link · ngoài C++	Làm môi trường đầu tiên để chạy capture SocketCAN thật.
T04 · A2 3% · Chưa làm	Viết virtual ECU và kịch bản vcan có ground truth Chưa có virtual ECU harness	Cấp nguồn kiểm thử live có thể đo và tái lập.
T05 · A3 3% · Chưa làm	Chọn adapter và chốt sơ đồ điện CAN lab BOM/wiring/driver · chưa chốt hardware	Tránh nhầm lỗi phần cứng thành lỗi AI hoặc SocketCAN.
T06 · A3 3% · Chưa làm	Bring-up can0 và xác nhận đường nhận vật lý capture(can0) [218] · chưa test vật lý	Cho phép thay nguồn vcan bằng CAN vật lý sau khi đường nhận đạt.
T07 · A4 3% · Chuẩn bị	Kiểm tra OS và build native trên Pi CMake/native build · VM khác Pi ARM64	Tạo runtime đúng thiết bị để chạy replay và capture.
T08 · B1 2% · Kiểm local	Parser và source adapter CSV parse_frame/read_csv [98/122]	Làm baseline kiểm tra xử lý dữ liệu độc lập phần cứng.
T09 · B1 4% · Chuẩn bị	Receiver SocketCAN và quan sát frame bị bỏ capture/recvmsg [218/244]	Nối kernel CAN với feature/inference trên edge.
T10 · B2 2% · Một phần	Chốt clock và timestamp live timestamp lambda [235] · user space	Tính IAT, ordering và delay có cùng miền clock.
T11 · B2 2% · Kiểm local	Window engine 100 ms Extractor.advance/reset [66/62]	Làm đơn vị tính feature, inference và quyết định.
T12 · B3 3% · Kiểm local	Feature toàn cục v1 dùng chung train/runtime Extractor.push/finish [76/46]	Cấp input có cùng schema cho Python training và C++ inference.
T13 · B3 4% · Chưa làm	Feature theo từng ID/signal và ngữ cảnh Chưa có per-ID/signal schema v2	Cải thiện phát hiện thay đổi tinh vi và giải thích được dấu hiệu.

“Kiểm local” là scope PC đã có bằng chứng, không tự đồng nghĩa VM/Pi đã đủ gate. Phụ thuộc và tiêu chí chi tiết có trong docs/TASK_BREAKDOWN.vi.md; bảng này map trực tiếp để đọc code.

Source / trace: Mặc định số [dòng] thuộc src/main.cpp, trừ khi ghi file khác. Source snapshot được ghi ở trang cuối.

TRA CỨU · WBS / ARCH

Map task → code → outcome (14–26)

Task đánh theo vị trí kiến trúc, không phải thứ tự bắt buộc. Một hàm có thể hiện thực nhiều block. % là công sức tương đối; tổng 38 task bằng 100%.

Task / arch / effort	Công việc và implementation hiện tại	Outcome dùng để
T14 · B4 3% · Kiểm local	Export cây và inference C++ Forest constructor/score [145/169]	Chạy model trực tiếp trong executable C++ trên PC/Pi.
T15 · B4 2% · Chuẩn bị	Xác nhận score parity trên Pi verify_pi_bundle.py [14] · golden parity	Cho phép chuyển từ local inference sang kiểm thử edge thực.
T16 · B5 1% · Kiểm local	Rule baseline cho demo Detector.emit [195] · hard rules	Đối chứng rule-only và bảo đảm AI không phải nguồn quyết định duy nhất.
T17 · B5 2% · Một phần	Profile và rule đúng mạng lab Whitelist [153] / rate [195] chỉ toy	Giảm false alert và làm cảnh báo có ý nghĩa với testbench thực.
T18 · B6 3% · Kiểm local	Fusion rule/AI và debounce cảnh báo Detector.emit state transitions [198]	Biến score/rule thành trạng thái cảnh báo ổn định và kiểm chứng được.
T19 · B6 2% · Một phần	Tách health khỏi alarm và theo dõi mất dữ liệu NO_DATA/counters có; chưa tách health	Tránh hiển thị healthy hoặc mất lịch sử khi sensor ngừng nhận.
T20 · C1 2% · Kiểm local	Dashboard review replay serve.replay [19] / UI poll [24]	Review logic đầu cuối trước khi nối phần cứng.
T21 · C1 3% · Chưa làm	Dashboard nguồn CAN live Chưa có live capture API/UI	Quan sát vận hành Pi khi đang nhận CAN thật.
T22 · C2 2% · Một phần	Chuẩn hóa event/alert contract emit JSON [206] · window event v1	Cho UI, logger và evaluator đếm/correlate cùng một sự kiện.
T23 · C2 4% · Chưa làm	Evidence và retention Chưa evidence buffer/retention	Từ một alert xem lại được frame liên quan, trong ngân sách lưu trữ đã chốt.
T24 · C3 3% · Một phần	Báo cáo chất lượng nhận diện report.py / synthetic validation report	Căn cứ quyết định model/profile có đủ dùng cho lab hay không.
T25 · C3 3% · Chưa làm	Benchmark tài nguyên và thời gian trên Pi emit inference_us [193]; chưa Pi bench	Chốt tải đáp ứng được và ngân sách tài nguyên edge.
T26 · C4 3% · Chưa làm	Bounded queues và logger bất đồng bộ Chưa queues/async C++ logger	Giữ đường nhận ổn định khi model/UI/logger có độ trễ.

“Kiểm local” là scope PC đã có bằng chứng, không tự đồng nghĩa VM/Pi đã đủ gate. Phụ thuộc và tiêu chí chi tiết có trong docs/TASK_BREAKDOWN.vi.md; bảng này map trực tiếp để đọc code.

Source / trace: Mặc định số [dòng] thuộc src/main.cpp, trừ khi ghi file khác. Source snapshot được ghi ở trang cuối.

TRA CỨU · WBS / ARCH

Map task → code → outcome (27–38)

Task đánh theo vị trí kiến trúc, không phải thứ tự bắt buộc. Một hàm có thể hiện thực nhiều block. % là công sức tương đối; tổng 38 task bằng 100%.

Task / arch / effort	Công việc và implementation hiện tại	Outcome dùng để
T27 · C4 2% · Một phần	Fault injection và phục hồi test_pipeline negative cases [60]	Xác nhận lỗi không bị diễn giải thành normal và có quy trình recovery.
T28 · D1 3% · Chưa làm	Chốt và lấy nguồn dữ liệu đại diện Chưa dữ liệu thực trong pipeline	Cung cấp đầu vào có provenance để train và đánh giá.
T29 · D1 3% · Chưa làm	Adapter dataset và kiểm tra chất lượng dữ liệu Chưa converter HCRL/ROAD/lab	Bảo đảm model học dữ liệu hợp lệ và evaluator có ground truth.
T30 · D2 3% · Một phần	Split theo phiên và chống data leakage prepare [75] · synthetic sessions	Giữ test thực sự chưa được dùng để fit/tune.
T31 · D2 2% · Một phần	Tạo feature dataset có version mode features [264] / prepare.extract [61]	Tái lập training và phát hiện schema mismatch khi deploy.
T32 · D3 2% · Kiểm local	Train baseline Isolation Forest normal-only prepare IsolationForest.fit [81]	Đưa anomaly detection vào pipeline C++ thay vì chỉ rule.
T33 · D3 3% · Một phần	Calibration và chọn candidate trên validation prepare threshold quantile [84]	Chọn operating point trước khi mở final test.
T34 · D3 4% · Chưa đạt	Đánh giá holdout và xử lý các case miss Failure report: payload 1/40 ML windows	Chặn việc đẩy model còn yếu lên môi trường sử dụng thực.
T35 · D4 2% · Một phần	Model bundle và metadata phiên bản Forest header/manifest [prepare 86/119]	Nhận diện chính xác bộ model/runtime được review hoặc rollback.
T36 · D4 3% · Kiểm local	Đóng gói chuyển sang Pi và kiểm integrity package_pi.py + verifier [14]	Giảm lỗi copy thủ công và tạo một gói review có thể xác minh.
T37 · D5 2% · Chuẩn bị	Chạy Pi staging và lưu biên bản review Guide/staging gates; chưa Pi staging	Xác định phần nào thực sự đạt trên Pi trước khi vận hành dài hạn.
T38 · D5 3% · Chưa làm	Service, rollback và cổng chấp thuận production Chưa service/updater/rollback code	Vận hành dài hạn với bản đã kiểm chứng và có đường phục hồi.

“Kiểm local” là scope PC đã có bằng chứng, không tự đồng nghĩa VM/Pi đã đủ gate. Phụ thuộc và tiêu chí chi tiết có trong docs/TASK_BREAKDOWN.vi.md; bảng này map trực tiếp để đọc code.

Source / trace: Mặc định số [dòng] thuộc src/main.cpp, trừ khi ghi file khác. Source snapshot được ghi ở trang cuối.

CHUNG · EXECUTION PLAYBOOK

Tra nhanh: lệnh nào đi qua phần nào?

Lệnh	Đường xử lý	Kết quả
cmake --build build	Compiler/linker; không gọi main runtime	Binary mới; không tự train model
ai-vids features INPUT OUTPUT	CSV → Extractor → vector	Feature CSV; không score/rule/state
ai-vids score MODEL FEATURES	Vector có sẵn → Forest.score	Scalar mỗi dòng; không capture/window
ai-vids replay INPUT MODEL	CSV → full core → emit	NDJSON; rx_drops/ignored mặc định 0
ai-vids capture vcan0 MODEL	Linux socket → full core	NDJSON live; timestamp nhận user space
python3 tools/serve.py	HTTP → subprocess replay khi bấm Play	API/UI replay; không tự thấy capture terminal
ai-vids run --config ...	CLI V2 đề xuất	Chưa có trong binary hiện tại; chỉ dùng sau khi implement

Tự xem dữ liệu trung gian, không train lại

```
./build/ai-vids features data/demo.csv runtime/inspect.csv
./build/ai-vids score models/demo.forest runtime/inspect.csv
./build/ai-vids replay data/demo.csv models/demo.forest
```

Bốn câu hỏi để đọc một event

- Nguồn là replay hay capture, timestamp thuộc clock nào?
- Feature/model/profile version nào tạo score và threshold?
- Reason đến từ rule hay ML; state có đang latch theo lịch sử không?
- Health/data_quality có đủ tin cậy, hay đang có gap/drop/storage lỗi?

Source / trace: CLI dispatch main.cpp:260; fields emit:206; code walkthrough docs/VM_EXECUTION_MAP.vi.md.

KẾT THÚC SPEC · V1.0

Nguồn tham chiếu và checklist review

Nguồn chính thức	Phần hỗ trợ trong spec
Linux Kernel — SocketCAN	Interface/socket API, vcan, controller modes
scikit-learn — IsolationForest	Quy ước score/offset và mô hình forest
systemd.service / systemd.exec	Type/restart và môi trường thực thi service
Python — http.server	Phạm vi server review; không dùng nguyên bản cho production
Raspberry Pi — remote access	SSH/SCP/tunnel cho quản trị local
Ubuntu — OpenSSH	Thiết lập SSH nếu dùng để copy/kết nối VM

Sơ đồ worker, requirement, release gates và config/CLI V2 là thiết kế đề xuất của dự án, không phải tính năng đã tồn tại chỉ vì có trong tài liệu. Tài liệu tham chiếu có thể theo bản mới; cần đối chiếu OS/kernel/systemd đích.

Thông tin cần bạn chốt để triển khai V2

- Pi RAM/OS/kernel; adapter/model controller, driver, bitrate và ID/FD coverage.
- Dataset/profile/DBC và quyền truy cập dữ liệu đại diện cho mạng đích.
- Recall/false-alarm/detection-delay budget, peak load và resource/storage quota.
- Readiness/recovery, evidence retention, release/rollback triggers và người chấp thuận.

> Review theo mã: ví dụ “PI-BOOT-03 cần heartbeat compute; T19 phải giữ alarm khi mất traffic”. Chỉ thực hiện production sau quyết định riêng của bạn.

Source snapshot: src/main.cpp SHA-256
`a7c99ba88fc043add530ec4f07fcb6804dda28e23934bb382eec40c52f3c3878`

Evidence dùng trong spec: docs/VALIDATION.vi.md, docs/REVIEW_PLAN.json, runtime/vm-log-walkthrough.json; bản source và gói đã trao đổi. User paste có 52 window cuối đầy đủ, không phải toàn bộ bộ hồ sơ kiểm thử VM.

Revision: 1.0 · 01/11/2026 · Nội dung tiếng Việt · Hai cấu hình V1/V2 trong một tài liệu.